

Modulo 1: Nivel de aplicacion. Tema 3: Correo Electronico

Estructura

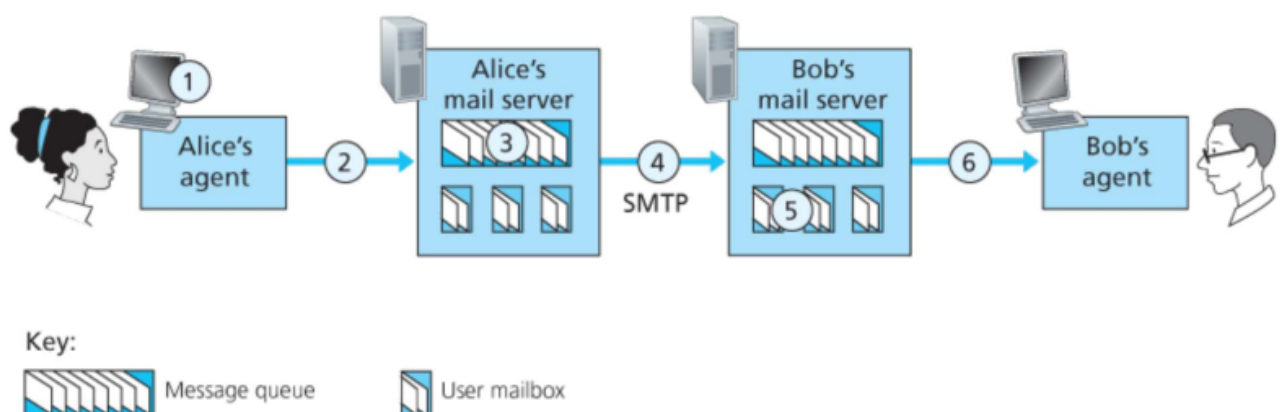
Existen tres componentes principales para el sistema de correo electronico:

- **Usuario** (User Agent): la aplicacion desde la que consultas el correo electrónico, como Google Chrome (luego te conectas a Gmail, Outlook, Webmail, ...) o la app de Correo del iPhone. Se conecta al servidor de correo para enviar y recibir mensajes.
- **Servidor de correo** (Mail server): servidor que almacena todos los correos electrónicos y los organiza para todos los usuarios registrados. Por ejemplo, mis correos personales están en los servidores de Gmail, los de las cosas de la uni están en los servidores de la UAM (@estudiante.uam.es), etc.
Tienen la característica de utilizar colas de mensajes, tanto para recibir correos como para enviarlos.
- **SMTP**: protocolo con el que se comunican los distintos servidores de correo electrónico.

Es importante observar que, a diferencia de lo que ocurría con HTTP, siempre es necesario utilizar dos servidores: uno es el remitente (en el que inicias sesión y redactas el correo) y otro es el destinatario (en el que la otra persona inicia sesión y lee el correo que se le ha enviado). En HTTP, el cliente (Google Chrome) mandaba la petición directamente al servidor web que le interesaba

SMTP: Puerto 25

SMTP nació en 1982 para el envío de correo y se creó mucho antes de HTTP. Asentó las bases de cómo nos comunicamos hoy en día desde nuestros dispositivos. Ejemplo Supongamos que Alice quiere mandar un correo a Bob



1. Alice redacta un nuevo correo, indicando la dirección del destinatario: bob@estudiante.uam.es, y pulsa "Enviar".
2. La aplicación envía el correo al servidor. Se almacena en la cola de los mensajes pendientes por enviar
3. El servidor saca de la cola el mensaje, consulta el destinatario y crea una conexión TCP con el servidor estudiante.uam.es
4. Utilizando el protocolo SMTP, se envía el correo

5. estudiante.uam.es recibe el correo en su cola de mensajes pendientes por distribuir. Una vez lo saca de la cola, ve que es para Bob y lo coloca en su bandeja de entrada
6. Bob se conecta al servidor y ve que tiene un correo nuevo

Funcionamiento del protocolo

De la misma manera que hemos hecho peticiones HTTP, vamos a mandar un correo desde la terminal. Primero, hay que instalar **Postfix**, un programa que te monta automáticamente un servidor de correo SMTP en tu ordenador, como Apache para HTTP. De esta manera, puedo enviarme correos electrónicos en localhost puerto 25 Conectándome con netcat (nc localhost 25), puedo enviar un correo electrónico:

In []: %%bash

```
nc localhost 25 <<EOF
HELO alejandroubuntu
MAIL FROM:<emisor@direccionemisor.es>
RCPT TO:<noexiste@alejandroubuntu>
RCPT TO:<alejandroo313@alejandroubuntu>
DATA
esto es un mensaje

esto es otro mensaje
adios
.
QUIT
EOF
```

```
220 alejandroubuntu ESMTP Postfix (Ubuntu)
250 alejandroubuntu
250 2.1.0 Ok
550 5.1.1 <noexiste@alejandroubuntu>: Recipient address rejected: User unknown in local recipient table
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as 8E9D6505528
221 2.0.0 Bye
```

Los mensajes del servidor siguen la siguiente sintaxis: **[Codigo de respuesta][Texto explicativo]**

Codigos de respuesta normales:

- 220 al conectar
- 250 tras comandos validos
- 550 si un destinatario no existe
- 354 despues de DATA para empezar a escribir el mensaje

Los mensajes del cliente siguen esta sintaxis: **[Comando] ([Argumento])** . La lista de comandos se puede mirar en internet. Los comandos usados en el ejemplo son:

- **HELO alejandroubuntu** : Identificación del cliente ante el servidor. Es importante porque es el primer paso de cualquier sesión SMTP. El servidor puede rechazar la conexión si no recibe este saludo
- **MAIL FROM:<emisor@direccionemisor.es>** : Indica la dirección de correo del remitente. Formato obligatorio: la dirección debe ir entre '<>'
- **RCPT TO:<alejandroo313@alejandroubuntu>** : Especifica un destinatario del mensaje. Se puede enviar el mismo mensaje a múltiples destinatarios. Si el usuario existe en el sistema y el servidor lo permite, será aceptado

- **DATA** : Le dice al servidor "voy a empezar a escribir el contenido del mensaje". Después de este comando, puedes escribir el cuerpo del mensaje. Terminas el mensaje con un punto (.) solo en una línea, lo que indica el final del cuerpo.
- **QUIT** : Termina la sesión de SMTP de forma limpia. El servidor normalmente responde con algo como 221 Bye.

Al principio del mensaje puedes utilizar **From: [direccion del emisor]**, **To: [direccion del destinatario]** o **Subject: [Asunto del correo]** y el servidor del destinatario podrá reconocer estas directivas.

Para leer el correo, por como funciona Postfix en Ubuntu, hay que ir al directorio `/var/mail/[usuario]`.

In [2]: `!cat /var/mail/alejandroo313`

```
From emisor@direccionemisor.es Sat Mar 8 12:07:11 2025
Return-Path: <emisor@direccionemisor.es>
X-Original-To: alejandroo313@alejandroubuntu
Delivered-To: alejandroo313@alejandroubuntu
Received: from localhost (localhost [127.0.0.1])
    by alejandroubuntu (Postfix) with SMTP id 96E9750D743
    for <alejandroo313@alejandroubuntu>; Sat, 8 Mar 2025 12:03:08 +0100 (CET)
Message-Id: <20250308110623.96E9750D743@alejandroubuntu>
Date: Sat, 8 Mar 2025 12:03:08 +0100 (CET)
From: emisor@direccionemisor.es
```

```
hola.
hola2
hola3
```

```
From emisor@direccionemisor.es Fri Apr 18 14:54:56 2025
Return-Path: <emisor@direccionemisor.es>
X-Original-To: alejandroo313@alejandroubuntu
Delivered-To: alejandroo313@alejandroubuntu
Received: from alejandroubuntu (localhost [127.0.0.1])
    by alejandroubuntu (Postfix) with SMTP id 8E9D6505528
    for <alejandroo313@alejandroubuntu>; Fri, 18 Apr 2025 14:54:56 +0200 (CEST)
Message-Id: <20250418125456.8E9D6505528@alejandroubuntu>
Date: Fri, 18 Apr 2025 14:54:56 +0200 (CEST)
From: emisor@direccionemisor.es
```

```
esto es un mensaje
```

```
esto es otro mensaje
adios
```

Se pueden ver dos mensajes ya que se hicieron dos pruebas.

Comparacion con HTTP

- Desde la perspectiva del cliente, SMTP es un **protocolo push** (solo te conectas para enviar algo al servidor) y **HTTP es pull** (te conectas para pedir informacion al servidor).
- **SMTP solo maneja lenguaje ASCII de 7 bits**. Es decir, cuando quieres enviar algo que no es ASCII, como tildes o imagenes, el servidor emisor lo codigoca a ASCII y el servidor receptor lo decodifica.
- SMTP tiene que enviar todo el **contenido comprimido en un mensaje**. En HTTP cada objeto (imagenes, css, html, ...) se envia en una respuesta separada

Protocolos de acceso al correo

Como hemos visto, SMTP nos permite mandar y recibir correos que incluso podrian tener asunto o imagenes. Sin embargo, a la que rascamos un poco, vemos que realmente asi no es como funcionan los sistemas modernos como Gmail o Outlook. Por ejemplo, ¿que pasaria si tengo el ordenador apagado? El servidor no estaria funcionando y no se podria enviar el correo.

Para solucionar esto, hoy en día el usuario esta separado de su servidor de correo. Antes, para leer y escribir correos tenias que acceder al servidor como hemos hecho en el ejemplo. Rapidamente se dieron cuenta de que esto no era efectivo, asi surgen los primeros protocolos de acceso al correo POP e IMAP.

Estos protocolos se centran en gestionar la bandeja de entrada y pedir al servidor los correos recibidos; es decir, son protocolos pull. Por esto, HTTP tambien se utiliza mucho como protocolo de acceso al correo.

POP: puerto 110

POP1 nace en octubre de 1984 y destaca por lo sencillo que es. Hoy en día, desde 2008 (fue hace 15 años...) para ser exactos, tenemos POP3. Junto con IMAP, es un protocolo muy extendido en los servidores de correo. Sin embargo, su simpleza conlleva una funcionalidad limitada.

Veamos un ejemplo con Dovecot, un paquete de linux encargado de preparar al servidor de correo de tu ordenador (el que hemos instalado en el ejemplo de SMTP) para peticiones POP3 e IMAP.

Una vez instalado, me conecto con nc localhost 110

In []: %%bash

```
nc localhost 110 <<EOF
user alejandroo313
pass <contraseña>
LIST # Lista los mensajes que hay en el servidor junto con su tamaño
RETR 1 # Recupera el mensaje 1
DELE 1 # Elimina el mensaje 1
RETR 1
RSET # Restablece el estado de la sesión
RETR 1
DELE 1
QUIT # Cierra la sesión (borra los mensajes marcados para borrar)
EOF
```

```
+OK Dovecot (Ubuntu) ready.
+OK
+OK Logged in.
+OK 2 messages:
1 475
2 513
.
+OK 475 octets
Return-Path: <emisor@direccionemisor.es>
X-Original-To: alejandroo313@alejandroubuntu
Delivered-To: alejandroo313@alejandroubuntu
Received: from localhost (localhost [127.0.0.1])
    by alejandroubuntu (Postfix) with SMTP id 96E9750D743
    for <alejandroo313@alejandroubuntu>; Sat,  8 Mar 2025 12:03:08 +0100 (CET)
Message-Id: <20250308110623.96E9750D743@alejandroubuntu>
Date: Sat,  8 Mar 2025 12:03:08 +0100 (CET)
From: emisor@direccionemisor.es

hola.
hola2
hola3
.
+OK Marked to be deleted.
-ERR Message is deleted.
+OK
+OK 475 octets
Return-Path: <emisor@direccionemisor.es>
X-Original-To: alejandroo313@alejandroubuntu
Delivered-To: alejandroo313@alejandroubuntu
Received: from localhost (localhost [127.0.0.1])
    by alejandroubuntu (Postfix) with SMTP id 96E9750D743
    for <alejandroo313@alejandroubuntu>; Sat,  8 Mar 2025 12:03:08 +0100 (CET)
Message-Id: <20250308110623.96E9750D743@alejandroubuntu>
Date: Sat,  8 Mar 2025 12:03:08 +0100 (CET)
From: emisor@direccionemisor.es

hola.
hola2
hola3
.
+OK Marked to be deleted.
+OK Logging out, messages deleted.
```

IMAP: puerto 143

Pero, yo en Gmail puedo hacer más cosas que leer y borrar mensajes. Por ejemplo, puedo crear carpetas de correos (Spam, Destacados, ...) que se compartirán a través de mis dispositivos y tengo un buscador para encontrar correos según palabras clave. Para aportar estas características nace IMAP en 1986. Pero claro, todo esto implica más complejidad en la implementación del protocolo.

Principalmente, IMAP asocia cada mensaje a una carpeta: cuando llega un correo nuevo, se va a INBOX (carpeta por defecto). Ahora, IMAP te ofrece comandos para crear nuevas carpetas, renombrarlas e incluso compartirlas entre usuarios. Toda esta estructura se crea en el lado del servidor.

Otras funcionalidades interesantes son la posibilidad de buscar correos utilizando palabras clave y poder obtener líneas específicas, algo útil cuando trabajas con conexiones lentas y solo quieres cargar texto y no algún video adjunto.

Y, ¿por qué no actualizar POP en vez de crear un protocolo nuevo? Porque la filosofía de IMAP es completamente distinta: el acceso al servidor se hará de manera dinámica. Mientras que POP solo se

conecta al servidor para pedirle un correo y se desconecta inmediatamente, IMAP va a permanecer conectado desde que pases la fase de autenticación. Esto permite notificar al usuario inmediatamente al recibir un nuevo correo y, aunque te conectes simultáneamente desde diferentes dispositivos, los cambios se cargarán en tiempo real.

HTTP: puerto de indias

Conforme la tecnología avanza, más gente empezó a entrar a su correo desde un navegador web y el tráfico de los clientes para comunicarse con sus servidores de correo pasó a ser HTTP (aunque los clientes usen protocolos como POP, IMAP o HTTP, los servidores de correo se comunican entre sí usando SMTP).

De esta manera, cuando entras a tu cuenta de Gmail desde Google Chrome, mandas una petición HTTP al servidor de correo y este te contesta (con una o más respuestas, ventajas del HTTP) con tu bandeja de entrada. Además, cuando quieras mandar un mensaje, mandarás un mensaje HTTP con el correo electrónico y el servidor te lo mandará. Es decir, está pensado como un app Web.

Sin embargo, los móviles de repente se hicieron súper potentes y pasamos de entender internet como un ente detrás de nuestro navegador a descentralizarlo en muchas aplicaciones. De hecho, probablemente la mayoría de veces que utilices tu correo desde la app de tu smartphone y todas estas vuelven a funcionar con últimas versiones de POP e IMAP.